

Tasker — Implantação no Kubernetes (Minikube) com Helm

Aplicação: Tasker — gerenciador de tarefas (to-do) multiusuário

Aluno: Jakson H. Z.

Orquestração: Kubernetes (Minikube) · Helm · Ingress NGINX

Este documento descreve **(a)** a aplicação e seus componentes/containers, **(b)** o roteiro de testes e **(c)** os artefatos Kubernetes (Deployment, Service, Secret, ConfigMap, PV/PVC e Ingress) utilizados na implantação da aplicação no Minikube.

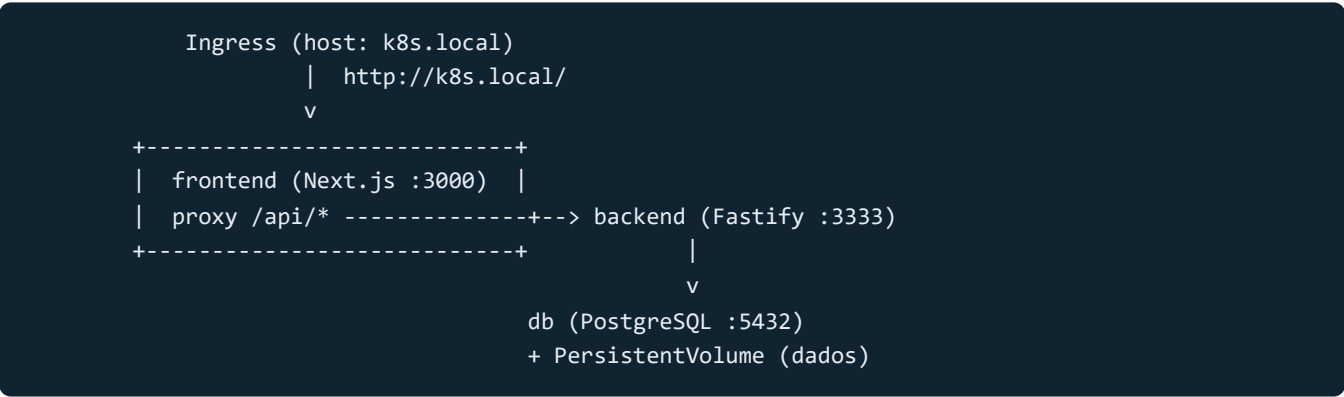
1. A aplicação e seus componentes

O **Tasker** é uma aplicação web de lista de tarefas (estilo Trello/To-Do) com autenticação por JWT. É composta por **três contêineres**:

Componente	Tecnologia	Porta	Imagem	Exposto externamente
frontend	Next.js 14 (React)	3000	<code>jaksonhz/tasker-frontend</code>	sim (via Ingress)
backend	Fastify + Prisma (Node.js 20)	3333	<code>jaksonhz/tasker-backend</code>	não (interno)
db	PostgreSQL 16	5432	<code>postgres:16-alpine</code>	não (interno)

Comunicação entre os componentes

O **único ponto de entrada público** é o frontend. O navegador nunca fala diretamente com o backend: as chamadas de API saem do navegador como caminhos relativos `/api/*` e são repassadas, **no servidor Next.js**, para o backend interno (recurso `rewrites` do Next, em `next.config.mjs`). O backend acessa o PostgreSQL pelo nome do serviço (`tasker-db`).



Vantagens dessa topologia no Kubernetes:

- **um único Ingress** (só o frontend é publicado);
- **sem CORS** (navegador e API na mesma origem `k8s.local`);
- backend e banco permanecem isolados como serviços `ClusterIP`.

Modelo de dados

O backend usa Prisma sobre PostgreSQL com as entidades **User**, **ListTODO**, **ItemTODO** e **Category**. As migrações são aplicadas automaticamente no start do backend (`npx prisma migrate deploy`).

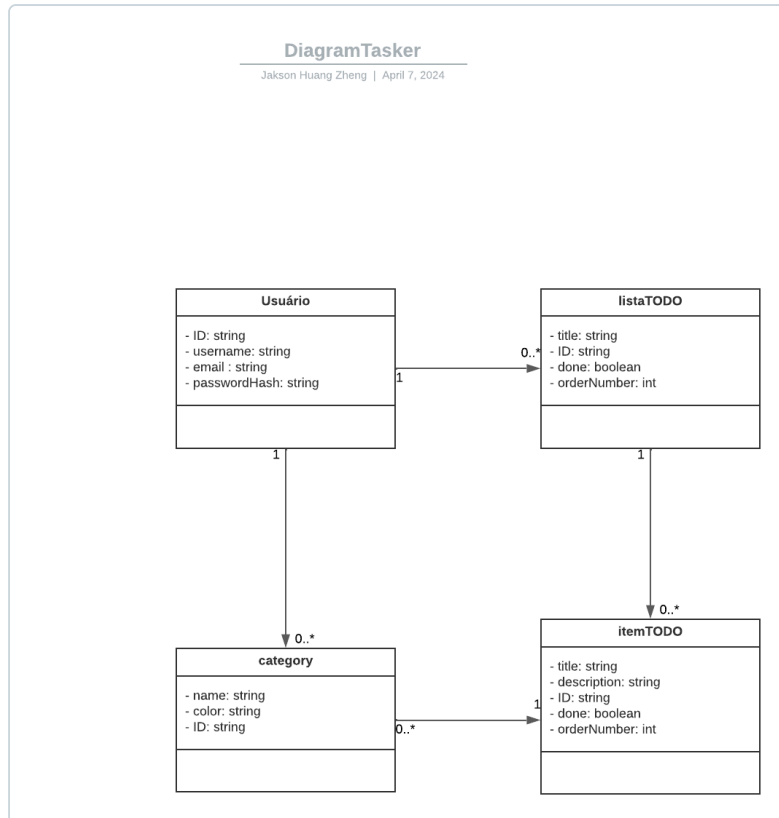


Diagrama de entidades do Tasker (Prisma / PostgreSQL).

2. Imagens Docker

As imagens da aplicação são construídas a partir dos `Dockerfile` de cada serviço (multi-stage build) e publicadas no **Docker Hub**:

- `docker.io/jaksonhz/tasker-frontend:latest`
- `docker.io/jaksonhz/tasker-backend:latest`
- `docker.io/postgres:16-alpine` (imagem oficial, sem build)

O script `scripts/build-images.sh` automatiza o build e a exportação para o Minikube (`minikube image load`) e, opcionalmente, o `docker push` para o Docker Hub.

O frontend recebe o build-arg `BACKEND_URL=http://tasker-backend:3333` , que embute no proxy `/api` o nome do serviço do backend dentro do cluster.

3. Artefatos Kubernetes

A implantação utiliza os seguintes objetos do Kubernetes:

Artefato	Nome	Função
Deployment	tasker-db	Contêiner do PostgreSQL
Deployment	tasker-backend	API (com <code>initContainer</code> que espera o banco)
Deployment	tasker-frontend	Contêiner do Next.js
Service (ClusterIP)	tasker-db	Expõe o banco internamente (5432)
Service (ClusterIP)	tasker-backend	Expõe a API internamente (3333)
Service (ClusterIP)	tasker-frontend	Expõe o frontend (3000) — alvo do Ingress
Secret	tasker-db-secret	Credenciais do PostgreSQL
Secret	tasker-backend-secret	<code>DATABASE_URL</code> e <code>SECRET_JWT</code>
ConfigMap	tasker-backend-config	Config não sensível (<code>PORT</code>)
ConfigMap	tasker-frontend-config	<code>BACKEND_URL</code> (destino do proxy <code>/api</code>)
PersistentVolume	tasker-db-pv	Volume <code>hostPath</code> (1Gi) para os dados do banco
PersistentVolumeClaim	tasker-db-pvc	Reivindicação do volume, montada no banco
Ingress	tasker-ingress	Publica a aplicação em <code>http://k8s.local</code>

Por que cada artefato

- **Deployment** — gerencia os Pods, garante as réplicas e reinicia em caso de falha. O backend tem um `initContainer` (`wait-for-db`) que aguarda a porta 5432 antes de subir, evitando *crash-loop* enquanto o PostgreSQL inicializa.
- **Service (ClusterIP)** — dá um nome DNS estável a cada camada, permitindo a comunicação interna mesmo quando os Pods são recriados.
- **Secret** — dados sensíveis (senha do banco, `DATABASE_URL`, segredo do JWT), injetados via `envFrom: secretRef`.
- **ConfigMap** — configuração não sensível, injetada via `envFrom: configMapRef`.
- **PersistentVolume / PersistentVolumeClaim** — fazem os dados do PostgreSQL **sobreviverem** à recriação do Pod (`storageClassName: manual`, `hostPath`, `ReadWriteOnce`). O banco usa `strategy: Recreate`.
- **Ingress** — roteia o host `k8s.local` para o Service do frontend; único objeto que expõe a aplicação para fora do cluster.

Os artefatos existem de duas formas equivalentes: **manifestos crus** em `k8s/` (`kubectl apply -R -f k8s/`) e **Helm Chart** em `tasker-chart/` (recomendado).

4. Helm Chart

O `tasker-chart/` é um **chart guarda-chuva (umbrella)** que agrega três subcharts, um por camada:

```

tasker-chart/
├─ Chart.yaml      # declara os subcharts (db, backend, frontend)
├─ values.yaml     # valores GLOBAIS (nomes, portas, credenciais, host)
├─ templates/
├─ ingress.yaml   # Ingress (k8s.local -> frontend)
└─ charts/
  ├─ db/          # PostgreSQL: Deployment, Service, Secret, PV, PVC
  ├─ backend/     # API: Deployment (+initContainer), Service, Secret, ConfigMap
  └─ frontend/    # Next.js: Deployment, Service, ConfigMap

```

Valores compartilhados em `values.yaml` sob a chave `global`:

```

global:
  ingress:
    host: k8s.local
    className: nginx
  db:      { name: tasker-db, user: docker, password: docker, database: tasker, port: 5432 }
  backend: { name: tasker-backend, port: 3333, jwtSecret: supersecretjwt }
  frontend: { name: tasker-frontend, port: 3000 }

```

Instalação:

```
helm upgrade --install tasker ./tasker-chart
```

5. Ingress e acesso via k8s.local

O Ingress publica **um único host** apontando para o frontend:

```

spec:
  ingressClassName: nginx
  rules:
    - host: k8s.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: tasker-frontend
                port: { number: 3000 }

```

No Minikube, habilita-se o controlador com `minikube addons enable ingress`. Para `k8s.local` resolver, adiciona-se ao `/etc/hosts` a linha `<minikube ip> k8s.local`. A partir daí, `http://k8s.local` serve o frontend e as chamadas `/api/*` são proxyadas ao backend.

6. Roteiro de testes

6.1 Pré-requisitos

```
bash scripts/install-tools.sh # Docker, kubectl, Minikube e Helm (usa sudo)
# reabra o terminal (grupo docker) ou rode: newgrp docker
```

6.2 Build e exportação das imagens

```
minikube start --driver=docker
bash scripts/build-images.sh # build + minikube image load
# opcional: docker login && bash scripts/build-images.sh --push
```

6.3 Implantação

```
bash scripts/deploy.sh # minikube + ingress + helm upgrade --install
echo "$(minikube ip) k8s.local" | sudo tee -a /etc/hosts
```

6.4 Verificação da infraestrutura

Passo	Comando	Resultado esperado
Pods	<code>kubectl get pods</code>	3 Pods em Running (1/1)
Services	<code>kubectl get svc</code>	3 ClusterIP
Ingress	<code>kubectl get ingress</code>	tasker-ingress com host k8s.local
Secret/ConfigMap	<code>kubectl get secret,configmap</code>	2 Secrets + 2 ConfigMaps
Volume	<code>kubectl get pv,pvc</code>	Bound

6.5 Teste de conectividade

```
curl -I http://k8s.local # 200 OK (frontend pelo Ingress)
curl -s -o /dev/null -w "%{http_code}\n" \
  http://k8s.local/api/user/list # 401 (backend via proxy /api)
```

Ou o **smoke test** automatizado (conectividade + registro + login com JWT):

```
bash scripts/smoke-test.sh
# 200 (/) · 401 (/api/user/list) · 201|409 (registro) · 200 + access_token (login)
```

Para acessar pelo navegador do Windows (WSL2), encaminhe o Ingress: `bash scripts/port-forward.sh` e abra `http://k8s.local:8080` (adicione `127.0.0.1 k8s.local` ao hosts do Windows).

6.6 Teste funcional (interface)

1. Abra `http://k8s.local` no navegador.
2. Em **Registrar**, crie uma conta (e-mail, usuário, senha).
3. Faça **login**.
4. Crie uma **lista** e adicione **itens/tarefas**.
5. Marque uma tarefa como concluída e recarregue a página.

Esperado: navegação funciona, cadastro/login retornam tokens e os dados permanecem após o reload (persistidos no PostgreSQL).

6.7 Teste de persistência (PV/PVC)

```
kubectl delete pod -l app=tasker-db
kubectl rollout status deployment/tasker-db
```

Esperado: após o Pod voltar, os dados criados continuam disponíveis (o volume persistente preservou o banco).

6.8 Inspeção do banco (opcional)

```
kubectl exec -it deploy/tasker-db -- psql -U docker -d tasker -c '\dt'
# lista as tabelas User, ListTODO, ItemTODO, Category
```

7. Comandos úteis

```
kubectl logs deploy/tasker-backend
kubectl describe ingress tasker-ingress
helm template tasker ./tasker-chart | less
helm lint ./tasker-chart
bash scripts/undeploy.sh           # helm uninstall tasker
bash scripts/undeploy.sh --purge   # também apaga os dados (PV/PVC)
minikube stop ; minikube delete
```